Using A Discrete Time Probabilistic Attack Simulator To Generate Temporally Labeled Training Data For GNN Based Node State Prediction

Ayush Kuril
kurilayush.24@kgpian.iitkgp.ac.in
Indian Institute of Technology Kharagpur
India

Shreyash Pattnaik
shreyashbbsr.2006@kgpian.iitkgp.ac.in
Indian Institute of Technology Kharagpur
India

Abstract

Intrusion detection systems (IDSs) traditionally rely on reactive traffic analysis and signature-based identification, leaving a critical vulnerability window between initial breach and detection. This paper proposes a proactive, Anomaly-based Intrusion Detection System (AIDS) leveraging Graph Neural Networks (GNNs) and a Discrete-Time Markov Chain simulator. By modeling enterprise networks as attack graphs enriched with CVE/CVSS data and firewall logic, our framework shifts from reactive analysis to forecasting node states (SAFE, EXPOSED, or COMPROMISED) at the next time-step. We demonstrate that structural attack graph topology contains predictive information regarding future compromise that cannot be explained by node-level features alone.

Keywords

Intrusion Detection System, Graph Neural Network, Markov Chain, Attack Graph, Cybersecurity

1 Introduction

There has been a significant uprise in cyber attacks targeting critical infrastructures, including finance transaction platforms, public transportation systems, healthcare data centers, and enterprise data. A Report by the World Economic Forum 2025 states that:

- 54% of large organizations see supply-chain interdependencies as their biggest barrier to building true cyber resilience.
- Nearly 60% report that shifting geopolitical dynamics have forced them to rethink and refine thdeir cybersecurity strategies.
- 1 in 3 CEOs now considers cyber espionage and the theft of sensitive information or IP a top concern.
- 66% expect artificial intelligence to play a major role in shaping cybersecurity in 2025, yet only 37% have put proper safeguards in place for the AI tools they are already using.

Therefore, it has become imperative to adeptly identify intrusions and safeguard crucial assets from malicious activities. An Intrusion Detection System (IDS) is such a system that can identify intrusive behaviors and raise alerts indicating intrusions are happening in computer networks. We propose an Anomaly-based Intrusion detection system built on statistics, expert knowledge, and machine learning (ML) which can identify anomalous behaviors, overcoming the limitation of Signature-based IDS. Among AIDS, ML-based IDSs stand out, characterized by intelligently extracting patterns and identifying anomalies from large amounts of data. Facing complex scenarios, ML-based AIDS may also falsely classify behaviors as intrusions, resulting in false positive alerts that seek further manual analysis by security experts.

A graph is a well-formatted data structure that can represent and extract complex patterns of data. Characterized by strong data representation, expressive and learnable inter-dependencies, graphs are widely used in social media, chemistry, biology, computer communication, etc. Recently, Graph Neural Networks (GNNs) have been gaining more attention due to the ability to effectively learn the hidden representation of node and topological information of a network represented as graph-structured data. More recently, some works have also been done to study the explainability of GNNs in this particular field. Moreover, the abundant information in computer networks such as vulnerabilities is not fully utilized to train GNN models, which could to some degree make the GNN models' predictions (un)certain and (un)explainable.

As hackers are becoming more sophisticated, equipped with advanced attacking skills, the situation necessitates highly robust IDSs designed to detect evasions. An attack graph is a modelling framework that depicts correlated multi-stage attacks in computer systems and networks and can be used to analyze potential attack paths that an adversary could take by exploiting vulnerabilities in networks. This paper implements a Graph Neural Network based Intrusion Detection System (GNN-IDS) along with a markov based time simulator in which the attack graph is adopted to represent the structural information and static attributes of computer networks, real-time measurements are used to simulate the dynamics in networks, and Graph Neural Networks are employed as the inference engine for intrusion detection.

1.1 Problem Statement

Security teams in an enterprise environment are protecting devices - they are reacting to things that have already happened. Security teams are alerted after an attacker compromises a host and begins traversing the network. That time between the initial breach and the notification of the attacker is where most damage occurs, such as the theft of passwords, deletion of data, or installation of ransomware. Intrusion detection systems do not shorten that window of time if they rely on specific signatures or machine learning; they merely read the data and categorize it, they don't predict the future.

This paper outlines three specific technical problems:

(1) Simple machine learning does not take network component connections into account. Standard systems treat each observation as a distinct list of characteristics. For example, when the model uses logistic regression or a gradient boosted tree, it treats host B as independent and alone. It does not acknowledge that host A was compromised two steps before when the attacker reached it through a Server Message Block exploit.

(2) Multi-stage attacks (as they are the primary method of serious network compromise) consist of precisely this step-by-step attack traversal. Rarely does an attacker jump directly from the internet to a database server. Rather, they compromise the web server, move to the application server, then compromise the database. Since simple machine learning does not see the relationship, it cannot learn to identify attack movements.
(3) Current work using Graph Neural Networks for intrusion detection is also reactive and requires a large amount of data. Recent approaches using Graph Neural Networks construct models from network flows.

1.2 Paper Overview

Our paper has a single core claim: That there exists information regarding future compromise within the topology of the attack graph (who is compromised and the CVE-weighted path topology between them and their neighbors) that cannot be explained by a model working solely with node-level features.

GAT learns this propagation through its attention-weighted message passing along the exploit edges, while the baselines working with the same features without graph structure do not. We use the three-class classification, temporal split, and hard accuracy as methodological decisions that attempt to verify this claim.

In this paper we walkthrough 5 sections. Firstly, the introduction of the problem statement which sets the context of rising cyber attacks against critical infrastructure. It introduces the core concept: a Graph Neural Network-based Intrusion Detection System (GNN-IDS) that uses attack graphs and a Markov simulator to proactively predict attacks. Then, we move on to the Problem Statement that highlights that current security and Paper Overview States the core claim: attack graph topology holds predictive information about future compromises that a Graph Attention Network can learn. Secondly, we go through the System Overview where we summarize the pipeline: generating a synthetic network, running a Markov simulator to flip "weighted coins" on exploits, and feeding the resulting node states into a GAT for prediction. Third, there is the Dataset Construction, which describes how the training data is generated by running the simulator over 150 synthetic graphs. And then we have our Experimental Setup where we briefly outline the hardware and environment, dataset splits, and baseline models (Logistic Regression, XGBoost) used for evaluation and move finally to Conclusion, Real Life Applications, and Limitations/Future Work.

1.3 System Overview

A synthetic enterprise network is built with role-based hosts across four firewall-separated zones, and real CVEs from the NVD are matched to each host's services to form a directed attack graph where every edge carries a CVSS-derived Bernoulli exploit probability.

A discrete-time Markov simulator runs multiple independent attack trajectories through each graph, flipping a weighted coin on every exploit edge at every timestep and recording the full node state snapshot—SAFE, EXPOSED, or COMPROMISED—before and after each step. Every non-Attacker node at every timestep becomes one row in the dataset, described by 29 features spanning static host identity, vulnerability severity, dynamic simulation state, and edge-aggregated attack pressure, with the label being that node's state at the very next timestep. The dataset is split by both graph identity and time—earlier timesteps train the model and later timesteps test it—then fed into a GAT that reads the exploit edge structure alongside node features to predict which nodes will transition state before the next exploit attempt lands.

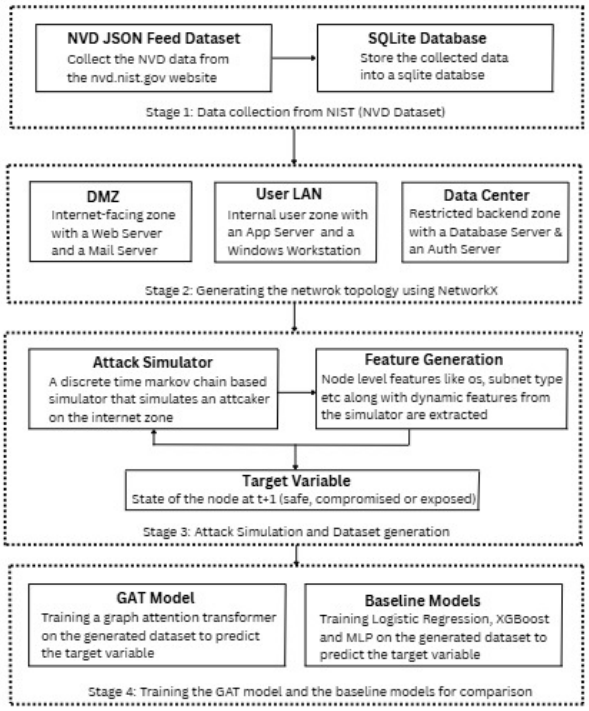


Figure 1: System Architecture Flowchart detailing the data flow from the Network Generator through the Markov Simulator into the final GAT Model.

1.4 Attack Graphs & Network Model

An attack graph is a directed acyclic graph representation of all possible paths of attack by exploiting the vulnerabilities in a network. Here the attack graphs have 4 logical layers corresponding to respective network zones, starting from Internet (Attacker origin) to DMZ (Web Servers, Mail Servers) to User LAN (App Servers, Windows Workstations) and finally Data Center (DB Servers, Auth Servers, Backup Servers). Generally, a graph can be expressed in the form $G = (V, E)$, where $v \in V$ refers to vertices or nodes and $e_{ij} = (i, j) \in E$ are directed links or edges pointing from source node v_i to destination node v_j.

Edges that were blocked by firewalls are excluded entirely. The weights w_{ij} carried by the edges signifying the exploit probability are calculated with a sigmoid function centered at a CVSS score of 6:

$$\text{base} = \sigma\left(\frac{\text{CVSS} - 6}{2.5}\right) \tag{1}$$

Table 1: Attributes of nodes in the attack graph

Attribute	Type	Description
id	int / str	Unique host identifier (or "Attacker")
ip	str	Synthetic IP in 192.168.x.y range
subnet	str	Internet, DMZ, User_LAN, Data_Center
role	str	Functional role (Web Server, etc.)
os	str	Operating system: "Linux" or "Windows"
services	list[dict]	Entry: name, port, protocol
vulns	list[dict]	Entry: cve, cvss, service, port, protocol
compromised	bool	Runtime state — True if node is owned

Table 2: Attributes of edges in the attack graph

Attribute	Type	Description
vuln_id	str	CVE identifier (e.g., CVE-2023-XXXX)
service	str	Exploited service name
port	int	Target port number
protocol	str	"TCP" (all services in this model)
weight	float	Exploit success probability [0.01, 0.98]
cvss	float	CVSS v3 base score driving probability
type	str	"exploit" for all attack graph edges
firewall	str	Name of the firewall that evaluated this edge

If an exploit is available, base $\times 1.4$, else base $\times 0.6$. The final weight is clamp(base, 0.01, 0.98).

This topology graph also has non-exploitable edges for visualization purposes: *exploit* (CVE-backed attack path), *physical* (Attacker to DMZ), and *lateral* (bidirectional Windows edges).

We generate the aforementioned Attack graphs by the Network generator class which constructs a directed multi-subnet enterprise network using `networkx`. It partitions hosts across three subnets: 20% DMZ, 50% User LAN, 30% Data Centers.

1.4.1 Firewall Modelling. Three stateful firewall instances enforce inter-zone traffic policy using a first-match, default-deny rule evaluation model:

- **FireWall1 (Internet to DMZ):** Allows HTTP:80, HTTPS:443, SMTP:25 inbound. Denies SSH:22, RDP:3389, SMB:445.
- **FireWall2 (DMZ to User LAN):** Allows HTTP:80, Java:8080, SSH:22. Blocks RDP:3389 and SMB.
- **FireWall3 (User LAN to Data Center):** Allows MySQL:3306, MSSQL:1433, LDAP:389. Blocks SSH and RDP.

1.5 Discrete Time Markov Simulation

An attack graph is static in nature and doesn't tell us how fast something will happen over time. We have implemented a discrete time simulator on the attack graph using the concept of a "Discrete Time Markov Chain" (DTMC). It follows the Markov property, where the next state depends solely on the current state.

Table 3: Inter-subnetwork Connectivity Rules

Source	Destination
Web Server	App Server
Mail Server	App Server
App Server	Database Server
App Server	Auth Server
Windows Workstation	Auth Server
Windows Workstation	App Server
Auth Server	Database Server
Auth Server	Backup Server
Database Server	Backup Server

Every node has 3 states:

- **COMPROMISED:** The attacker has successfully exploited it.
- **SAFE:** No compromised node can reach it via an exploit edge.
- **EXPOSED:** At least one predecessor is compromised, meaning it is actively being targeted.

1.6 Exploit Probabilities

For a directed edge $(u \to v)$ with weight $p(u, v)$, the single attacker probability is computed as:

$$p = \text{clamp}\left(\sigma\left(\frac{\text{CVSS} - 6}{2.5}\right) \times k, 0.01, 0.98\right) \tag{2}$$

where $k = 1.4$ if a public exploit exists, and $k = 0.6$ if not.

For multiple attackers, node v may have compromised predecessors $u_1, u_2, \ldots, u_k$. The probability that v survives all of them is:

$$P(v \text{ survives step } t) = \prod_{i=1}^{k} (1 - p(u_i, v)) \tag{3}$$

Therefore the probability that v falls to at least one attacker is:

$$P(v \text{ compromised at step } t) = 1 - \prod_{i=1}^{k} (1 - p(u_i, v)) \tag{4}$$

At the start of every simulation run, every edge weight is perturbed by uniform noise to ensure stochastic behavior at the run level:

$$\hat{p}(u, v) = \text{clamp}(p(u, v) + \mathcal{U}(-0.1, +0.1), 0.01, 0.98) \tag{5}$$

1.7 Graph Neural Networks from a IDS POV

1.7.1 Node-Level IDS as Graph Classification. We reframe the IDS challenge as a three-class node-level classification problem on a temporal graph. At every discrete timestep t, the IDS classifies every host v into one of three security states:

$$y_v^{(t+1)} \in \{0 \text{ (safe)}, 1 \text{ (exposed)}, 2 \text{ (compromised)}\}. \tag{6}$$

1.7.2 Input Representation. Each training sample contains a node feature matrix $X \in \mathbb{R}^{N \times 25}$. For each node v, the feature vector x_v is constructed as:

$$x_v = [\text{sn}(v) \parallel \text{role}(v) \parallel \text{os}(v) \parallel \text{vuln}(v) \parallel \text{str}(v) \parallel \text{dyn}(v) \parallel \text{edge}(v)] \tag{7}$$

Algorithm 1 Discrete-Time Attack Simulator

Input: G — attack graph: nodes (hosts) and directed exploit edges
$p(u, v)$ — exploit success probability derived from CVSS
T — max timesteps (default $T = 20$)
Output: history — list of per-step snapshots

Exploit probability formula:
base = sigmoid((CVSS − 6)/2.5)
where sigmoid$(x) = 1/(1 + e^{-x})$
$p(u, v)$ = clamp(base × 1.4, 0.01, 0.98) **if** public exploit
$p(u, v)$ = clamp(base × 0.6, 0.01, 0.98) **otherwise**

procedure SIMULATE(G, T, seed)
  for each edge (u, v) in G **do**
    add random noise ▷ $\sim \mathcal{U}(-0.1, 0.1)$, clamp to $[0.01, 0.98]$
  end for

  set state(v) ← COMPROMISED for Attacker/Internet nodes
  set state(v) ← SAFE for all remaining nodes
  history ← empty list

  for $t = 1$ **to** T **do**
    attackers ← all nodes currently in state COMPROMISED
    newly_compromised ← empty list

    for each u in attackers **do**
      for each neighbour v of u in G **do**
        if state(v) = COMPROMISED **then**
          skip ▷ already owned
        end if
        draw random number $r \sim$ Uniform$(0, 1)$
        if $r < p(u, v)$ **then**
          add v to newly_compromised ▷ exploit succeeded
        end if
      end for
    end for

    for each v in newly_compromised **do**
      set state(v) ← COMPROMISED
    end for

    for each node v with state(v) = SAFE **do**
      if any predecessor of v is in state COMPROMISED **then**
        set state(v) ← EXPOSED
      end if
    end for

    append { step: t, states_before, states_after } to history

    if all nodes reachable from Attacker are COMPROMISED **then**
      break ▷ early stop; no further spread possible
    end if
  end for

  return history
end procedure

Edge index $E \in \mathbb{Z}^{2\times|E|}$ and Edge attributes $\in \mathbb{R}^{|E|\times 3}$ carry the weight, CVSS, and exploit availability flag.

1.7.3 Graph Attention Network Architecture. We compute a scalar coefficient α_{uv} through GAT attention mechanism for every edge (u, v). The attention score extends the standard formulation to include edge attributes:

$$e_{uv} = \text{LeakyReLU}\left(\mathbf{a}^T[\mathbf{W}h_u \parallel \mathbf{W}h_v \parallel \mathbf{W}_e \cdot \text{edge\_attr}_{uv}]\right) \quad (8)$$

$$\alpha_{uv} = \frac{\exp(e_{uv})}{\sum_{k\in N(v)} \exp(e_{kv})} \quad (9)$$

The node representation from layer l using multi-head attention with concatenation is:

$$h_v^{(l)} = \Big\|_{k=1}^{K} \text{ELU}\left(\sum_{u\in N(v)} \alpha_{uv}^k \cdot \mathbf{W}^k \cdot h_u^{(l-1)}\right) \quad (10)$$

1.7.4 Loss Function and Optimization. Training uses Negative Log-Likelihood Loss with class weighting:

$$\mathcal{L} = -\frac{1}{N}\sum_v w_{y_v} \cdot \log P(y_v \mid x_v, G) \quad (11)$$

We use Adam with weight decay. Seven hyperparameters are optimized using Optuna's TPE sampler with MedianPruner.

2 Dataset and Case Study

The dataset for this problem statement is constructed by running the Discrete Time Simulator on a large collection of independently generated graphs. The goal here is to produce a node level, temporally labelled dataset where each row or sample encodes the state of a network host i.e. a node at a single point in time, with the label being what state the node transitions to at the next timestep. This date construction helps us to formulate the intrusion detection problem as a node level multi class classification problem between SAFE, EXPOSED and COMPROMISED nodes.

The generation pipeline is as follows: 150 independently generated attack graphs, 5 simulations with each graph by jittering the edge weights and up to 20 timesteps per run. Now to make sure there is structural diversity, the number of hosts in the attack graphs changes from between 20, 30 and 40 in a round robin schedule across all the 150 graphs. This means the model is exposed to networks of varying scale during training rather than always seeing the same sized topology. Every graph is generated with a deterministic seed making the entire dataset fully reproducible.

At each timestep of each simulation run, one feature row is extracted per non-Attacker node. The Attacker node is excluded entirely as it is always COMPROMISED and is never a prediction target. Additionally, SAFE nodes that have no incoming exploit edges are skipped, since they can never be reached by the attacker and would only inflate the SAFE class artificially, worsening class imbalance without contributing useful training signals.

The target variable or the label for each node is `y_next_state`. This is the state of the node at timestep $t + 1$, captured from the simulation at the following step. This means that our model learns to predict the next state of the node by learning about its neighbourhood and current step. Before any rows are extracted from a graph, the pipeline checks that the attack graph contains at least one exploit edge. If the CVE sampling happens to assign no vulnerabilities to any node, the graph is skipped entirely. In practice this is rare but the check prevents degenerate graphs from polluting

the dataset. The approximate total dataset size before filtering is:

$$150 \text{ graphs} \times 5 \text{ sims} \times 20 \text{ steps} \times \sim 25 \text{ nodes} \approx 375,000 \text{ rows}$$

The exact count varies because some simulations terminate early (when all reachable nodes are compromised) and some nodes are filtered out as described above. Now there are a total of 25 features that we extract from these graphs, some static features while some are dynamic in nature. The subnet, role and OS belonging to the nodes is one hot encoded. Apart from these one hot encoded features there are some static features like `vuln_count`, `max_cvss`, `in_degree` etc. Then lastly there are some dynamic features like `steps_since_exposed`, `avg_cvss_in` etc. which are determined via the discrete time simulator. Table 4 briefly discusses what each feature in the dataset encodes.

The target column `y_next_state` takes one of three string values SAFE, EXPOSED, or COMPROMISED which are mapped to integer class labels $\{0, 1, 2\}$ respectively before being passed to the GNN. The class distribution is inherently imbalanced since most nodes remain SAFE for most of the simulation, particularly in early timesteps before the attacker has penetrated deep into the network. This class imbalance is addressed at training time through class-weighted loss computation.

The dataset is split at the graph level, not at the row or timestep level. This is because if we split by timestep then for example putting the last 30% of timesteps into the test set would allow the test set to contain late-stage snapshots of the same graphs the model trained on. The model would have already seen those graphs' topology, CVE assignments, and edge structure, making the test accuracy a measure of generalisation to new timesteps on known networks rather than to genuinely unseen networks. This constitutes data leakage. The Graph level split for train/validation/test is detailed in Table 5.

3 Experimental Results

3.1 Experimental Setup

Execution environment. All experiments are conducted on the HP OMEN 16 laptop equipped with an AMD Ryzen 7 7840HS octa-core processor (8 cores, base clock 3.8 GHz, boost up to 5.1 GHz), 16 GB DDR5 RAM, a 512 GB NVMe SSD, and an NVIDIA GeForce RTX 4060 discrete GPU (8 GB GDDR6). The source codes are implemented under Python environment (version 3.12.13) and the GNN models are built on the graph operators in `torch_geometric` (version 2.7.0) in PyTorch (version 2.10.0+cu128).

Datasets: Min-max normalization is applied to continuous features (`vuln_count`, `max_cvss`, `steps_since_exposed`, `max_weight_in`, `avg_weight_in`, `avg_cvss_in`). Training split parameters are used on the validation and test splits to avoid data leakage. One-hot encoded features (subnet, role, OS, current state) are not scaled as they are binary.

The dataset is node-level with one entry per non-Attacker node, for each retained timestep and simulation, provided the node has been compromised and the outcome is not SAFE throughout the whole simulation. Each row represents a training sample for flat baselines (LR, XGBoost). For the GAT, each group of rows belonging to the same (`graph_id`, `sim_id`, `timestep`) forms a single PyG Data object. Total dataset size statistics are presented in Table 6.

Models: The attack graph's four logical layers require the GAT to learn multi-hop scenarios. 3 GATConv layers are used, providing a 3-hop receptive field to cover the full depth of the attack graph. Input features for LR and XGBoost are node feature vectors, while the GAT takes graph-structured Data objects. Detailed architectures are available in Table 7.

Hyperparameters: The dataset is unbalanced (SAFE: 60%, EXPOSED: 25%, COMPROMISED: 15%). Class-weighted loss is used for the GAT and sample weights for XGBoost. Adam optimizer with weight decay is used for the GAT, tuned using Optuna. The final GAT is trained for up to 100 epochs, with ReduceLROnPlateau and gradient clipping at `max_norm` = 2.0.

3.2 Results

3.2.1 Baseline Study. To evaluate the effectiveness of the GNN model for an IDS system we decided to baseline test it against three classical machine and deep learning baseline models. We chose Logistic Regression, XGBoost, and a Neural Network on identical held out test graphs. All models are evaluated on two primary metrics: Accuracy (which is adjusted so that it accounts for only the SAFE and EXPOSED classes because once a class is COMPROMISED it remains as such and so predicting that is trivial and would falsely inflate the accuracy scores) and F1- EXPOSED.

$$\text{Acc}_{\text{hard}} = \frac{\text{Correct predictions on SAFE and EXPOSED nodes}}{\text{Total SAFE and EXPOSED nodes}} \tag{12}$$

More formally, excluding all nodes currently in the COMPROMISED state at prediction time:

$$\text{Acc}_{\text{hard}} = \frac{|\{v \in \mathcal{V}_H : \hat{y}_v = y_v\}|}{|\mathcal{V}_H|}, \quad \mathcal{V}_H = \{v \in \mathcal{V} : s_v \neq \text{COMPROMISED}\} \tag{13}$$

F1-EXPOSED

$$P_E = \frac{\text{TP}_E}{\text{TP}_E + \text{FP}_E}, \quad R_E = \frac{\text{TP}_E}{\text{TP}_E + \text{FN}_E} \tag{14}$$

$$\text{F1}_{\text{EXPOSED}} = \frac{2 \cdot \text{TP}_E}{2 \cdot \text{TP}_E + \text{FP}_E + \text{FN}_E} \tag{15}$$

where $\text{TP}_E, \text{FP}_E, \text{FN}_E$ are true positives, false positives, and false negatives for the EXPOSED class respectively.

As shown in Figure 2, the GNN achieves the highest accuracy of 88.2% outperforming the Logistic Regression (81.3%), XGBoost (81.2%), and the Neural Network (77.9%) by margins of 6.9, 7.0, and 10.3 percentage points respectively. More critically, the performance gap widens substantially on F1-EXPOSED, where the GNN achieves 73.8% compared to XGBoost (60.5%), Logistic Regression (58.3%), and Neural Network (56.7%) with improvements of 13.3, 15.5, and 17.1 percentage points respectively. This disproportionate advantage on F1-EXPOSED relative to accuracy is a key finding because it suggests that the structural and temporal inductive biases encoded in the GNN are especially well suited for identifying this transitional EXPOSED state, which is inherently relational to its neighbouring nodes and is short lived as well.

The normalised confusion matrices (Figure 3) offer deeper insights into how different models behave for different classes. The

Table 4: Complete Dataset Feature Extraction Breakdown

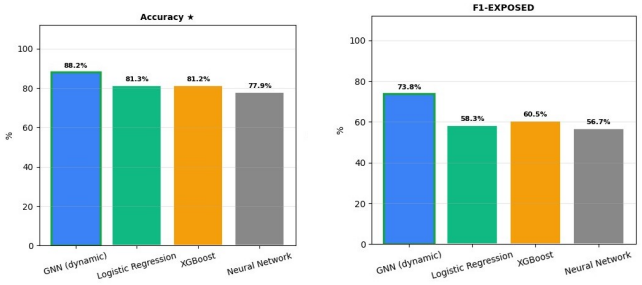
#	Feature	Description
1	`sn_internet`	1 if node is in the Internet zone, 0 otherwise
2	`sn_dmz`	1 if node is in the DMZ, 0 otherwise
3	`sn_lan`	1 if node is in the User LAN, 0 otherwise
4	`role_web`	1 if node is a Web Server, 0 otherwise
5	`role_mail`	1 if node is a Mail Server, 0 otherwise
6	`role_app`	1 if node is an App Server, 0 otherwise
7	`role_winws`	1 if node is a Windows Workstation, 0 otherwise
8	`role_db`	1 if node is a Database Server, 0 otherwise
9	`role_auth`	1 if node is an Auth Server, 0 otherwise
10	`role_backup`	1 if node is a Backup Server, 0 otherwise
11	`os`	Operating system of the host — 1 for Linux, 0 for Windows
12	`vuln_count`	Number of CVEs assigned to this host (0–3)
13	`max_cvss`	Highest CVSS v3 base score among assigned CVEs; 0.0 if none
14	`has_exploit`	1 if any assigned CVE has a known public exploit, 0 otherwise
15	`in_degree`	Number of incoming exploit edges to this node
16	`out_degree`	Number of outgoing exploit edges from this node
17	`state_safe`	1 if the node is in state SAFE at time t, 0 otherwise
18	`state_exposed`	1 if the node is in state EXPOSED at time t, 0 otherwise
19	`state_comp`	1 if the node is in state COMPROMISED at time t, 0 otherwise
20	`steps_since_exposed`	Number of timesteps elapsed since this node first became EXPOSED
21	`num_incoming_edges`	Total number of incoming exploit edges to this node
22	`max_weight_in`	Highest exploit success probability among all incoming exploit edges
23	`avg_weight_in`	Mean exploit success probability across all incoming exploit edges
24	`avg_cvss_in`	Mean CVSS v3 score of all CVEs associated with incoming exploit edges
25	`frac_exploit_in`	Fraction of this node's predecessors that are currently in the COMPROMISED state

Table 5: Graph-level Partition Split

Partition	Graphs	Purpose
Train	100	Model training (all timesteps used)
Validation	20	Early stopping and hyperparameter tuning
Test	30	Final evaluation (completely held out)

Table 6: Dataset split statistics

Split	Graphs	Snapshots	Node Rows
Train (folds 1-4)	120	~7,200	~103,000
Validation (fold 0)	30	~1,800	~26,000
Test (held out)	150	~2,250	~32,000
Total	**150**	**~11,250**	**~161,000**



(a) Accuracy Comparison **(b) F1-EXPOSED Comparison**

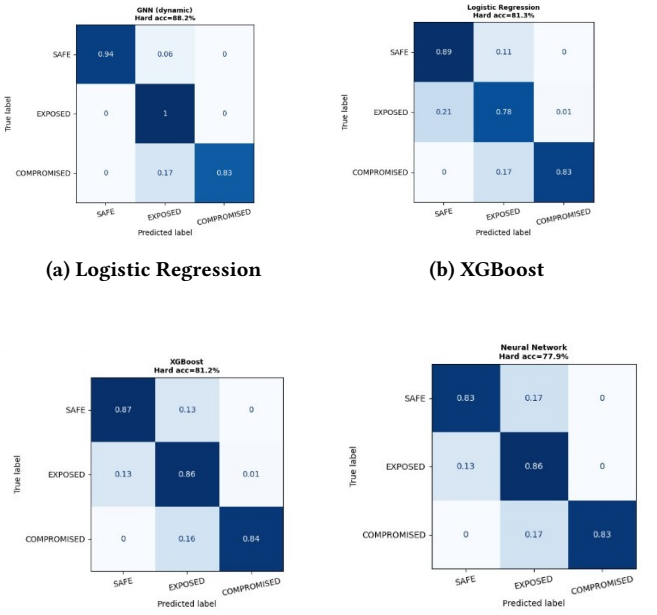
Figure 2: Performance metrics comparison between the GNN and baseline models.

recall of the GNN for exposed class is nearly 1 meaning it misses virtually no exposed nodes, which is a crucial property for an IDS. In contrast the baselines achieve EXPOSED recalls of 0.78 (Logistic Regression), 0.86 (XGBoost), and 0.86 (Neural Network), indicating a substantially higher miss rate. Across all models, SAFE recall is relatively strong (0.83–0.94), and COMPROMISED recall is broadly similar (0.83–0.84), suggesting that the distinguishing power of the GNN is concentrated in its superior handling of the EXPOSED class rather than in the majority or terminal classes.

One notable observation in the GNN confusion matrix is that 17% of the compromised nodes are misclassified as exposed, a phenomenon also observed in the baselines to some extent. This can be explained from the fact that a node has just transitioned from exposed to compromised, retains its structural and feature similarity to its prior state. The GNN's temporal message passing allows it to partially resolve this ambiguity, yet the complete separation of these adjacent states remains a challenge for all models which

Table 7: Model architectures. $D_{in} = 29$ (node feature dimension). K = number of attention heads. BN = BatchNorm.

Model	Layer 1	Layer 2	Output layer
LR	StandardScaler, Logistic(D_{in}, balanced)	-	Softmax(3)
XGBoost	XGBClassifier(D_{in}, depth{3,5,7})	300 estimators, early stop	Softmax(3)
GAT	GATConv(29 → 64, K=4, edge_dim=3) + BN + ELU	GATConv(256 → 64, K=4) + BN + ELU GATConv(256 → 64, K=1) + BN + ELU	Linear(64 → 32) Linear(32 → 3) LogSoftmax



(a) Logistic Regression **(b) XGBoost**

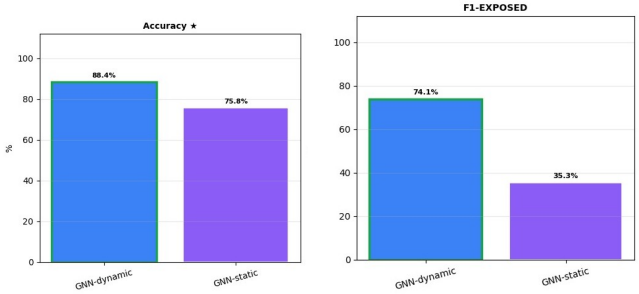
(c) Neural Network **(d) GNN Model**

Figure 3: Normalized Confusion Matrices across all evaluated models.

suggest that it is an intrinsic property of how the attack spreads rather than a modelling deficiency. Another notable observation is that logistic regression actually has better accuracy than XGBoost and neural network, this is mainly because the feature set is strongly engineered via the discrete time simulator and these basically linearly separable signals and simpler models like Logistic Regression often tend to match or even do better than complex models in such cases.

3.2.2 Ablation Study on GNN using dynamic and static features. Now to isolate and quantify the impact of the discrete time simulator on our proposed model, we have trained two separate GNN with same hyperparameter and architecture but one of them is trained on a single static snapshot of the data while the other gets the features extracted from discrete time simulator along with the static features of the graphs.

As shown in Figure above, removing the temporal features leads to a substantial degradation in the performance: accuracy drops from 88.4% to 75.8% (−12.68 percentage points), and F1-EXPOSED collapses from 74.1% to 35.3% (−38.73 percentage points). The magnitude of the F1-EXPOSED particularly striking as it tells us that the temporal features are not just merely incrementally helpful but are actually structurally essential for the exposed class detection.

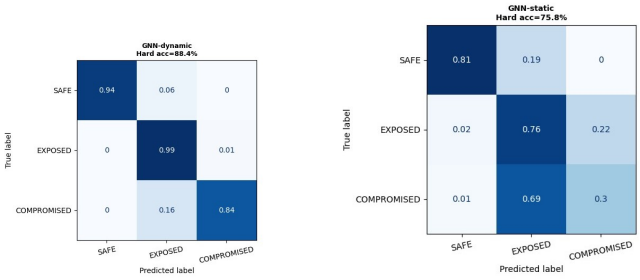


(a) Dynamic Features **(b) Static Features**

Figure 4: Ablation study comparing models trained with dynamic versus static features.

The confusion matrices (Figure 5) make the mechanism of the failure transparent. The GNN-static model achieves SAFE recall of 0.81 but shows severe degradation for the COMPROMISED class, with recall falling to just 0.30 meaning 70% of truly compromised nodes are misclassified, with 69% of COMPROMISED nodes incorrectly labelled as EXPOSED. Simultaneously the EXPOSED recall drops to 0.76 from a 0.99 in the dynamic model. This pattern reveals a systematic confusion between the exposed and the compromised states in the absence of the temporal features from the discrete time simulator. Without access to the trajectory of node state changes over time, the model cannot distinguish between a node that is currently in a transitional exposure state and one that has already progressed to full compromise. Both exhibit similar structural neighbourhood properties in any given static snapshot i.e they are both connected to infected neighbours, but their histories are fundamentally different.

These findings provide an explanation for the value of the discrete time simulator in this model. The temporal feature sequence encodes the speed and ordering of the neighbourhood state changes, enabling the model to disambiguate states that are topologically



(a) With Temporal Features **(b) Without Temporal Features**

Figure 5: Normalized Confusion Matrices demonstrating the structural necessity of temporal features.

indistinguishable in isolation. The +38.73pp improvement on F1-EXPOSED can therefore be attributed not to a richer graph architecture, but specifically to the temporal signal that allows the model to reason about when and how fast infection spread reaches a node.

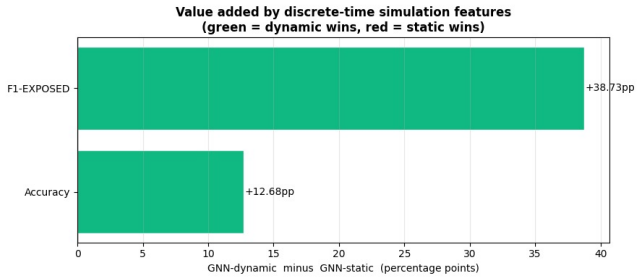


Figure 6: Value Addition metric tracking temporal signal effectiveness.

4 Conclusions

4.1 Real Life Applications

4.1.1 Proactive Threat Hunting and Vulnerability Prioritization. Perhaps the most immediate operational application is in Security Operations Centers (SOCs), where vulnerability scanner output must currently be manually prioritized by analysts, a task which is both tedious and lacks context. A CVSS 9.8 on a host in the Data Center appears to be identically risky to a CVSS 9.8 on a host in the DMZ on a flat vulnerability report. But the DMZ host's existence may permit a direct exploit path from the Attacker with an 0.85 edge weight, allowing it to immediately rise above any other, seemingly more vulnerable host. The $P(\text{COMPROMISED})$ per node at the next time step gives us exactly the context-dependent prioritization we need.

4.1.2 Real-time attack path monitoring. For enterprises running continuously monitored networks, this simulation engine can be fed current attack graph state information near real-time. Upon receipt of an EDR notification that a DMZ web server has been COMPROMISED, the corresponding node in the attack graph may be switched to COMPROMISED, and the GAT re-evaluated on the modified graph.

4.1.3 Incident response support. During an active attack, the most critical question is what the attacker will target next. Standard incident response relies on reactive forensics, analyzing logs after progress is made. This framework reverses this; using an attack graph of current compromises, the model predicts likely next targets with probability scores, allowing responders to contain attacks proactively.

4.2 Limitations

The simulator uses the Markov property such that the success of the exploit in each step only depends on the state, not on past history. Real world attackers are adaptive such that failures in exploits can be retried multiple times with various payloads. None of these are modelled here. The simulator also assumes the attacker "fires" every edge at every step; this happens at a far greater speed and parallelism than in real-world lateral movement.

The dataset is entirely synthetic. The CVEs are real and the topology realistic, however attack paths are generated by a probability model, not derived from any real-world recorded attacks. This is one of the most fundamental gaps from a real-world use-case. Furthermore, the present framework models no defender. The attack graph is static in its execution; no patching occurs mid-attack. Lastly, the framework only uses CVEs known at the time of generation; Zero-day exploits cannot be modelled.

4.3 Future Work

The most impactful extension would be to explore temporal GNN architectures. The current GAT models all steps as being isolated, whereas a GNN that combines multiple steps like a Graph Recurrent Neural Network could infer from repeated exposures. Secondly, defender agents can be modelled as a game where a defender GNN plays against an attacker GNN via Reinforcement Learning. Third, a real topology. Replacing the synthetic generator with real-world network topologies obtained via CMDB systems or Nmap would drastically improve the system's credibility.

4.4 Closing Remarks

In this paper we proposed a predictive intrusion detection framework that shifts from reactive traffic analysis to forecasting node states that are SAFE, EXPOSED, or COMPROMISED at the next time-step. This proactive approach provides security analysts with vital lead-time before attacks succeed. A key contribution is the EXPOSED state, representing the period where a node has compromised neighbors but isn't yet breached. Predicting transitions through this state offers a defensive window in contrast to binary Benign/Malicious models.